ScreenSync Digital Signage and Instagram Content Automation

Technical Project Report

Prepared 13 November 2026 | Repository state reviewed 13 November 2026

This report describes the implemented system and separates verified code behavior from configuration-dependent operation and proposed future work. It does not claim live Meta, S3, database, GitHub Actions, or television test results that have not been observed.

Contents

1. Executive summary
2. Project context and problem
3. Objectives, scope, and requirements
4. System architecture
5. Existing signage workflows
6. Data model and API design
7. Playback synchronization
8. Instagram automation
9. OCR and event-date handling
10. Security, reliability, and operations
11. Implementation status and verification
12. Limitations and risks
13. Alternatives and design rationale
14. Recommended academic extension
15. Conclusion

Appendix A. Terms

Appendix B. Configuration checklist

References

1. Executive summary

ScreenSync is a browser-based digital signage system for centrally managing content shown on multiple display devices. Administrators use a React dashboard to create playlists, schedule tasks, publish alerts, manage users, and inspect connected TVs. An Express backend stores application records in MongoDB and broadcasts playback updates to browser-based TV clients over WebSockets.

The core synchronization method gives all TVs the same playlist and a server-generated synchronization timestamp. Each client uses that timestamp and the ordered item durations to calculate which item should be playing at the current time. A display that joins late can catch up to the shared position instead of starting from item zero. Alerts and scheduled tasks take precedence over ordinary playlist playback.

The Instagram module is an optional extension. It is disabled by default and uses Meta authorization and Graph API calls, subject to account eligibility and granted permissions. It stores imported media in an S3-compatible object store, runs Tesseract.js OCR on image slides, saves extracted text and event metadata in MongoDB, and sends uncertain event dates to administrator review. Approved Instagram items can be merged into the existing active playlist, preserving the same WebSocket broadcasts and TV timeline.

The current implementation contains OCR and rule-based date handling, not a custom-trained machine-learning model. Syntax checks, focused date-classification assertions, dependency checks, and a frontend production build passed. Live Meta authorization, S3 access, MongoDB integration, scheduled GitHub Actions execution, and end-to-end TV playback were not validated with production credentials or devices. The OAuth code records token expiry but does not implement token refresh; this is a material production-readiness gap.

2. Project context and problem

2.1 Operational problem

A multi-display environment needs a single place to manage content and a reliable way to keep screens aligned. Manually updating each TV is slow and can leave displays showing different material. Network conditions also differ: a wired display may receive updates quickly, while a Wi-Fi display may experience delay or jitter. ScreenSync addresses centralized control, shared playback timing, live updates, and recovery when a TV reconnects.

2.2 Content lifecycle problem

College announcements and event posters become stale after their events. Removing them too early is also harmful, especially when a poster contains both an event date and a separate registration deadline. The Instagram extension addresses this lifecycle problem by retaining imported media, extracting readable text, recording event fields, requiring review when dates are ambiguous, and scheduling removal only after a confirmed event end date plus seven full days in Asia/Kolkata time.

2.3 Project framing

The system is web-based, but its technical contribution is not only a dashboard. Its core is a multi-client playback service with a shared timeline and real-time delivery. The Instagram extension adds a content-ingestion and human-review pipeline. For an academic submission, a defensible research contribution would be a trained event-information extraction model evaluated against the current OCR-and-rules baseline. That model and its evaluation are proposed work, not part of the implementation described here.

3. Objectives, scope, and requirements

3.1 Objectives

- Provide a browser-based administration interface for playlists, tasks, alerts, users, and device status.
- Deliver playlist and schedule changes to connected displays with low latency.
- Keep displays aligned using one server-issued timeline rather than per-device playlist clocks.
- Allow the same TV client to run in wired LAN and Wi-Fi display configurations.
- Prepare an optional Instagram integration that can remain disconnected without affecting existing signage.

- Download Instagram media to persistent storage and preserve the original order of carousel slides.
- Use OCR and conservative date rules to prevent uncertain content from being expired automatically.
- Archive expired media records rather than permanently deleting them.

3.2 Scope and boundaries

Included in the repository: React administration pages, Express REST APIs, JWT and role authorization, MongoDB/Mongoose models, WebSocket delivery, a shared browser TV client, Instagram OAuth/API service code, S3-compatible media storage code, Tesseract.js OCR, rule-based event-date review, and GitHub Actions workflow files.

Not established by the repository alone: Meta app approval or permissions for the college account, live object-storage credentials, token refresh, a custom ML model or benchmark, production schedule execution, or measured TV synchronization accuracy. Existing manually uploaded files still use the original local uploads directory; the new durable object-storage path is for Instagram media.

3.3 Functional requirements

- Only authorized administrators can modify operational content or Instagram settings.
- A reconnecting TV receives the current active playlist and schedule state.
- Instagram remains off by default and does not require Meta credentials for ordinary application operation.
- Duplicate Instagram IDs are skipped; scheduled runs record outcomes.
- Uncertain event end dates require human confirmation and cannot trigger automatic event expiry.
- Instagram items coexist with manual items and are distinguishable by source metadata.
- Expired records remain archived with OCR text, event data, source ID, and timestamps.

4. System architecture

4.1 Component responsibilities

- **React admin frontend:** login, dashboard, playlist/task/alert administration, device view, user management, and Instagram settings/review.
- **Express backend:** REST APIs, JWT and role enforcement, static dashboard/TV hosting, upload handling, Instagram orchestration, and media streaming.
- **MongoDB with Mongoose:** users, playlists, tasks, alerts, Instagram settings, imported-media records, event/OCR data, and sync history.
- **WebSocket service:** TV registration, heartbeats, connection state, playlist/task/alert broadcasts, and admin device updates.
- **Shared TV client:** connects to the backend, renders media, calculates playlist position, resolves priority content, and reports heartbeat state.
- **GitHub Actions:** triggers feed, story, and expiry endpoints on UTC schedules after the workflow files are published to the repository default branch.
- **S3-compatible storage:** persists imported Instagram media outside the backend instance filesystem.
- **Tesseract.js:** runs OCR on imported image buffers and returns text for storage and date analysis.

4.2 Request and data flow

1. An administrator signs in. The frontend stores the returned JWT and attaches it to protected API requests.
2. The backend validates the token and role, reads or writes MongoDB documents, and returns the application's JSON response format.
3. When a playlist is broadcast, the backend marks it active, assigns a synchronization timestamp, updates playback state, and sends it to connected TVs.
4. An Instagram schedule or administrator starts a sync. The backend verifies the scheduler secret or administrator identity, then calls Meta and processes eligible records.
5. Media bytes are downloaded from permitted Meta HTTPS hosts and written to private object storage. Metadata and OCR text are stored in MongoDB.

6. Approved Instagram playlist items are merged with manual items and sent through the same WebSocket service as ordinary playlists.

4.3 Deployment topology

In the department-PC deployment, one Windows computer runs the backend, MongoDB may run locally or remotely, and TVs open the shared TV page over the college network. In a hosted deployment, Render runs the backend and MongoDB is remote; Instagram media storage must be external and durable. The backend serves the dashboard, TV client, APIs, and WebSocket endpoint from one application server and port.

5. Existing signage workflows

5.1 Authentication and roles

Passwords are hashed with bcrypt before storage. Successful login returns a JWT. Authentication middleware verifies the token, loads the associated user, and attaches the identity and role to the request. Admin-only routes use role authorization. A viewer can inspect dashboard information but cannot change protected content.

5.2 Playlist lifecycle

A playlist is a named MongoDB document containing ordered items. An item specifies a media type, media URL or text, and duration. The model validates media types, required fields, text length, and duration bounds. The playlist also stores whether it is active and the timestamp from which its shared playback cycle is computed. Only one playlist is intended to be active at a time.

Administrators can create, edit, delete, and broadcast playlists. Editing an active playlist rebroadcasts its current content. Instagram items are tagged with source metadata so playlist edits preserve them and the Instagram service can refresh only its own subset.

5.3 Tasks and alerts

Tasks have a title, text and/or optional media, priority, and start/end schedule. Alerts have a message, priority, and expiry timestamp. The TV client resolves active priority content before falling back to the playlist. Alerts and tasks are therefore temporary layers over normal playback, not separate playlist systems.

5.4 Device status and reconnection

TV clients register with a device ID and report heartbeat state. The backend tracks online status and last heartbeat, then notifies admin clients when the device list changes. A reconnecting TV receives the current playback state, active playlist, and relevant schedule content, so it can recover without an administrator manually resending each item.

6. Data model and API design

6.1 Core MongoDB models

Model	Key information
User	Name, normalized email, bcrypt password hash, role, timestamps.
Playlist	Name, ordered playlist items, active flag, sync timestamp, creator, timestamps.
Task	Title, content/media, priority, schedule, creator, timestamps.
Alert	Message, priority, expiry, active flag, publisher, timestamps.
InstagramSettings	Singleton configuration, account identity, encrypted token, toggles, durations, last syncs, job locks.
InstagramMedia	Unique source Instagram ID, account, ordered slides, OCR text, event fields, review/status flags, story/removal/archive timestamps.
InstagramSyncRun	Job type, status, start/end time, imported/skipped/expired counts, and message/error summary.

Mongoose schemas define validation and indexes. Imported media has a unique Instagram ID and indexed status/removal fields. The settings model stores one default configuration record. The token field is excluded from normal query selection

and explicitly loaded by the service that needs it. Mongoose supports schema-level indexes and validation hooks. Mongoose schema guide

6.2 Representative API routes

Route group	Examples	Access
Authentication	POST /auth/login; GET /auth/me	Login or valid bearer token
Playlists	GET /getPlaylists; POST /createPlaylist; POST /broadcastPlaylist/:id	Read: authenticated; writes: admin
Playback	GET/POST /playback	Authenticated; control is admin-only
Devices	GET /getDevices	Authenticated
Instagram administration	GET /api/instagram/status; PATCH /settings; POST /sync	Admin JWT
Instagram review	GET /api/instagram/media; PATCH /media/:id/review; POST /media/:id/remove	Admin JWT
Instagram OAuth	POST /api/instagram/connect; GET /api/instagram/oauth/callback	Admin starts flow; callback validates state
Scheduled jobs	POST /api/instagram/jobs/:job	Shared scheduler-secret header
TV media stream	GET /api/instagram/media/:postId/:order	Read-only active, auto-inserted media

7. Playback synchronization

7.1 Shared timeline model

When a playlist is broadcast, the backend sets a timestamp slightly in the future using the synchronization buffer. All TVs receive the same playlist and timestamp. If item durations are d1 through dn, the cycle length is their sum. Each client computes elapsed time relative to the timestamp modulo the cycle, then walks the item durations to find the interval containing elapsed time.

```
cycleDuration = sum(item.duration)
elapsed = (currentTime - syncTimestamp) modulo cycleDuration
activeItem = first item whose cumulative duration contains elapsed
```

A future timestamp gives clients a short opportunity to receive the message and prepare. A late-joining client computes its current position from the same timeline. Wi-Fi mode preloads media where possible; LAN mode is treated as a lower-latency reference. This is best-effort browser synchronization, not a frame-accurate video-wall guarantee.

7.2 Pause, resume, and navigation

Playback control is shared through backend state. On pause, the server records when the pause began. On resume, the timeline shifts by the pause duration so playlist position remains frozen. Next, previous, and restart actions adjust the timestamp and rebroadcast the playlist so all displays recalculate from the same point.

7.3 Priority resolution

The client evaluates unexpired alerts first, then tasks whose schedules contain the current time, then the normal playlist. When priority content ends, the TV returns to its playlist timeline. This preserves one normal-playback schedule while supporting urgent interruptions.

8. Instagram automation

8.1 Disabled-by-default integration

Instagram settings live in a separate model. Automation, feed sync, story sync, and automatic playlist insertion default to false. The core app can start without Meta credentials. An administrator must configure Meta and storage settings, connect an eligible account, and explicitly enable automation.

8.2 OAuth and token storage

The current implementation uses a Facebook Login flow that expects an Instagram professional account discoverable through an authorized Facebook Page. The backend creates a random OAuth state, stores its SHA-256 hash for ten minutes, and verifies it on callback. It exchanges the authorization code, requests available pages and linked Instagram business accounts, and stores the selected account ID and username. The access token is encrypted with AES-256-GCM using a key derived from an environment secret. The code records an expiry if Meta returns one.

Token refresh is not implemented. Storing an expiry timestamp should not be confused with maintaining a long-lived connection. Before production operation, the Meta flow, required scopes, Graph API version, token lifetime, and refresh behavior must be confirmed against the actual app and account. Meta's API availability depends on account and permission eligibility. Meta Instagram API collection

8.3 Feed and story sync

Feed sync requests media fields including ID, caption, media type, media URL, permalink, timestamp, and carousel children. Story sync requests story IDs, media URLs, media type, and timestamp. The service follows pagination with a safety cap, retries selected transient API failures, and checks existing Instagram IDs before importing. Per-item failures are logged and recorded so one bad item does not necessarily stop the entire run.

8.4 Durable media and TV delivery

The storage adapter uses the AWS SDK for JavaScript and supports Amazon S3 or compatible endpoints. It generates deterministic object keys from a hash of the Instagram post ID and slide order. The bucket is expected to remain private. TV playlist entries contain ScreenSync media URLs, not Meta CDN URLs or S3 credentials. The backend streams stored objects from S3 to TVs. AWS SDK S3 examples

8.5 Carousel handling

A carousel is stored as one InstagramMedia record with an ordered array of slides. The service fetches carousel children, downloads supported image and video slides in returned order, OCRs image slides, and stores each slide's media type, object key, URL, order, content type, and OCR text. When inserted into the active playlist, slides become consecutive standard playlist items with the configured carousel duration. The TV client needs no separate carousel engine.

8.6 Playlist insertion and manual content

The playlist item schema tags imported items with source=instagram and the source post ID. Refresh logic replaces only the Instagram subset and retains manual items. Manual playlist editing also preserves imported items, preventing accidental deletion or conversion. An administrator can explicitly remove an imported item from the active playlist through the Instagram page; the record is archived, not permanently deleted.

9. OCR and event-date handling

9.1 OCR processing

Tesseract.js is initialized lazily and reused for image slides within a sync run. The OCR service returns recognized text, which is stored per slide and as combined post text. Video slides are stored and played but do not undergo OCR. The default language is English; additional language codes can be configured, subject to available Tesseract language data and deployment access. Tesseract.js wraps the Tesseract OCR engine and supports worker-based recognition in Node.js. Tesseract.js documentation

9.2 Current date-extraction policy

The date parser is rule-based. It recognizes a limited group of formats: day/month/year numeric dates, ISO-style year-month-day dates, and English month-name dates. It examines nearby OCR lines for event-related terms, registration language, or offer-expiry language. It also makes simple heuristic attempts to derive an event name, venue, and 12-hour time values. These heuristics are not a trained event-understanding model and may fail on unfamiliar layouts, languages, OCR errors, or unusual date formats.

OCR outcome	System behavior

OCR outcome	System behavior
One clearly labeled event date	Can set start and end to that date and calculate removal.
Two clear event dates in an event context	Can use the ordered dates as start and end.
Multiple or conflicting dates	Keeps the item in review; no automatic expiry.
Registration deadline only	Stores the deadline candidate and asks an admin for the event end date.
Offer expiry only	Stores an explicit expiry candidate but does not substitute it for event end.
No reliable date	Keeps the item pending review; it is not automatically removed.

9.3 IST retention and story expiry

For date-only event values, the implementation treats the supplied date as an Asia/Kolkata calendar date. The removal timestamp is set to midnight IST after seven full calendar days following the event date. The daily expiry job archives active records whose removal timestamp has passed and refreshes the playlist so TVs stop receiving the item. The S3 media remains stored.

Stories are handled independently. The implementation calculates story expiry as the Meta story timestamp plus 24 hours. The expiry job marks expired story records as story_expired and removes them from the active playlist refresh. This is the application's availability rule; actual story visibility still depends on Meta endpoint behavior and permissions.

10. Security, reliability, and operations

10.1 Security controls in the code

- JWT authentication and admin role checks protect operational and Instagram administration routes.
- Passwords are bcrypt-hashed and excluded from ordinary user document queries.
- Meta tokens are encrypted at rest with AES-256-GCM; the key is supplied through environment configuration.
- OAuth state is random, hashed in storage, time-limited, and checked with a timing-safe comparison.
- Scheduled job routes use a separate shared secret header, not an administrator JWT.
- Media downloads require HTTPS on allow-listed Meta hosts, limit redirects, and enforce byte caps.
- S3 objects are intended to be private; TVs receive only a ScreenSync streaming endpoint.
- Archived records and objects are retained rather than deleted by the expiry job.

10.2 Reliability mechanisms

The backend retries selected Meta API and media-download failures. Feed, story, and expiry work use database lock timestamps to reduce overlapping runs, while GitHub Actions concurrency groups serialize runs of each named workflow. Unique Instagram IDs prevent duplicate database records. Deterministic S3 keys make repeated uploads for the same source slide use the same key. Sync records store job status and item counts.

These controls are not a substitute for monitoring. GitHub scheduled workflows can be delayed, services can be unavailable, and expired Meta tokens can cause failures. Operators should inspect sync history and backend logs instead of assuming a cron trigger ran at its exact minute. GitHub documents that scheduled workflows use cron syntax and may be delayed under load. GitHub Actions workflow syntax

10.3 Configuration and deployment

The backend requires Node.js 20 or newer for the installed AWS SDK version, plus MongoDB. Ordinary local operation uses existing database and JWT settings. Instagram activation additionally requires Meta app ID and secret, exact redirect URI, Graph API version and scopes, token-encryption secret, scheduler secret, S3 bucket and region, and storage credentials. S3-compatible services can set an endpoint and path-style option. Tesseract language data must be reachable on the first OCR run unless provisioned locally.

The GitHub Actions workflow files are present in the local repository state. They do not run from a local checkout: they must be published to the GitHub default branch and configured with the required Actions secrets. The changes described in this report have not been pushed, so scheduled workflows are not active in GitHub.

11. Implementation status and verification

Subsystem	Status in reviewed code
Core admin and signage APIs	Existing implementation; frontend/backend source and routes are present.
TV WebSocket registration and playback	Shared client and broadcast service exist; not tested on live TVs in this review.
Instagram admin section and settings	Implemented in code; Meta account setup is not configured here.
Meta OAuth and Graph API discovery	Code implemented; actual credentials, permissions, and current API behavior not live-tested.
S3-compatible media storage	Adapter implemented; bucket, region, credentials, and object streaming not live-tested.
Image OCR and date rules	Tesseract.js integration and heuristic parsing implemented; no project-trained ML model.
Carousel processing	Ordered child download and playlist-slide mapping implemented; not validated against a live carousel.
Scheduled jobs	Workflow files and protected endpoints implemented locally; inactive until published to default branch.
Token lifecycle	Encryption and expiry recording implemented; automatic refresh is not implemented.
End-to-end validation	No complete Meta → S3 → MongoDB → WebSocket → TV test performed.

Checks performed

- Node syntax checks passed for changed Instagram backend modules, playlist controller, server, and shared TV client.
- Focused assertions passed for a clearly labeled event date, a separate registration deadline, an ambiguous multiple-date case, and the seven-day IST removal timestamp.
- AWS S3 and Tesseract.js dependencies resolved through npm at the versions recorded in the backend package files.
- The frontend Vite production build completed successfully.
- Git whitespace validation reported no diff-check errors.

These are code-level checks. They do not establish that Meta permissions, Render variables, S3 credentials, MongoDB behavior, GitHub Actions execution, or a specific television model work end to end.

12. Limitations and risks

- Meta permissions and endpoints vary by account type, app configuration, and platform rules; the integration has not been tested with the college account.
- Token refresh is absent. A token can expire after authorization, making later scheduled syncs fail until the account is reconnected or refresh support is added.
- The date parser is a conservative heuristic with limited formats and English-oriented cue words. It may send valid posters to review or misread text; human review is preferred to risky removal.
- Video reels and video carousel slides are stored and played but not OCR-processed.
- The S3 adapter has not been tested against a configured bucket. Bucket permissions, encryption policy, lifecycle policy, and costs still need operational setup.
- Existing manual uploads still use the local uploads directory. Only Instagram media uses the new object-storage flow.
- The TV media stream is a read-only route for active auto-inserted content without dashboard JWTs, because TVs do not sign in as admin users. Production deployment should use HTTPS and monitor access.
- Browser synchronization is best effort, not frame-accurate. Codecs, autoplay rules, clocks, network jitter, and rendering performance can affect playback.
- GitHub Actions schedules can be delayed and remain inactive until workflows are published and secrets configured.

13. Alternatives and design rationale

Decision	Why it fits this project	Alternative and trade-off
MongoDB with Mongoose	Matches the existing application and its nested playlist/media documents.	PostgreSQL offers relational constraints and reporting but requires migration and redesign.

Decision	Why it fits this project	Alternative and trade-off
WebSockets	Supports prompt broadcasts and live device status.	Polling is simpler but slower/noisier; MQTT suits larger fleets but adds a broker.
Browser TV client	One shared client runs on browser-capable displays and small PCs.	Native TV apps access platform features but require separate builds and maintenance.
One shared playlist timestamp	Every client derives position from one timestamp and ordered durations.	Per-device state offers customization but weakens global alignment.
Official Meta API	Uses supported authorization and API behavior rather than scraping.	Manual upload is simpler but loses automation; scraping is brittle and unsupported.
S3-compatible storage	Imported media persists beyond Render restarts and avoids temporary Meta URLs.	Render disk is convenient locally but is not durable cloud storage.
GitHub Actions	External scheduler invokes jobs without an in-process timer.	Render cron or a queue worker is an alternative with different operational ownership.
Tesseract.js and review	Provides OCR and keeps uncertain date decisions under human control.	Managed OCR may improve convenience but adds cost and external data processing.

There is no universally “only correct” architecture. These choices preserve the existing ScreenSync playback path and avoid ephemeral media storage. They are trade-offs, not proof that alternatives are inferior.

14. Recommended academic extension

14.1 Proposed contribution

A credible research extension is confidence-aware event-field extraction from college posters. It would compare the current Tesseract-plus-rules baseline with a trained model that labels event names, venue, dates, times, registration deadlines, and offer-expiry dates. The system should abstain when the event end date is uncertain and route the item to the existing administrator review queue.

14.2 Experimental plan

1. Build a permission-cleared, representative poster dataset and annotate target fields, date roles, and ambiguity labels.
2. Split training, validation, and test data by original poster/source to reduce near-duplicate leakage.
3. Use the current rule-based extractor as the baseline. Train a custom NER/date-role model and report field-level results.
4. Prioritize event-end-date exact match and false auto-approval rate, alongside precision, recall, F1, OCR error, review coverage, and processing latency.
5. Compare confidence thresholds: higher confidence can enable automatic approval; lower confidence requires human review.
6. Integrate model output without changing playlist synchronization or TV behavior.

A layout-aware model such as LayoutLMv3 could be explored if the dataset includes word bounding boxes and compute/time allow it. LayoutLMv3 uses both text and image information for document AI. It is a possible extension, not a feature of the current system. LayoutLMv3 paper

14.3 Why blockchain is not recommended

One organization administers these displays and content. There is no current multi-party trust problem requiring a distributed ledger. A conventional audit trail recording who approved or edited an event date is simpler, faster to query, and easier to operate. Blockchain would add key management, transaction overhead, and privacy concerns without improving event extraction or synchronization.

15. Conclusion

ScreenSync is a centralized, browser-based digital signage system with a persistent backend, live WebSocket communication, and a shared playback timeline for multiple displays. Its Instagram extension adds official-API discovery, encrypted token storage, durable media-storage support, OCR, conservative event-date review, carousel handling, and playlist integration through the existing architecture.

The implementation is a substantial full-stack and distributed-client system, but deployment and ML claims must remain precise. The integration needs actual Meta and storage configuration, token refresh, and end-to-end testing before production use. The current OCR/date logic is heuristic. A custom trained extraction model, accompanied by a labeled dataset and measured evaluation, is the strongest next step if the project needs an explicit ML contribution.

Appendix A. Key terms

Term	Meaning in ScreenSync
Active playlist	The single playlist currently sent to TVs for normal playback.
Global timeline	Shared synchronization timestamp plus ordered item durations used by all TVs.
Carousel	One Instagram post with multiple ordered child media items.
OCR	Optical character recognition: converting text visible in an image into machine-readable text.
Event end date	Confirmed last calendar date of the event; distinct from registration deadline or offer expiry.
Removal timestamp	Event end date plus seven full IST calendar days, represented as an absolute timestamp.
Review queue	Imported media whose event end date cannot safely be approved automatically.
S3-compatible storage	Object storage accessed through the S3 API, including Amazon S3 and compatible providers.
WebSocket	Persistent two-way connection used for broadcasts and live device updates.
Abstention	Choosing human review instead of making an uncertain automatic decision.

Appendix B. Configuration and deployment checklist

1. Use Node.js 20 or newer and configure MongoDB URI and a secure JWT secret.
2. For Instagram, set META_APP_ID, META_APP_SECRET, META_REDIRECT_URI, META_GRAPH_VERSION, and scopes supported by the Meta app.
3. Set INSTAGRAM_TOKEN_ENCRYPTION_KEY and INSTAGRAM_SCHEDULER_SECRET to separate strong random secrets. Preserve the encryption key securely.
4. Create a private S3 bucket and configure bucket, region, and AWS-compatible credentials. For compatible services, set endpoint/path-style options.
5. Choose Tesseract languages and configure media/image byte limits appropriate to Render memory.
6. Configure GitHub Actions secrets SCREENSYNC_URL and INSTAGRAM_SCHEDULER_SECRET, then publish workflow files to the default branch.
7. Configure the exact Meta redirect URI and verify the college professional account permissions.
8. Validate OAuth, token expiry behavior, feed/story access, object storage, OCR, admin review, expiry, and TV playback in staging.
9. Keep automation disabled until external services and credentials have been validated.

References

- Meta, Instagram API collection. https://www.postman.com/meta/instagram/documentation/6yqw8pt/instagram-api
- Amazon Web Services, Amazon S3 examples using SDK for JavaScript v3. https://docs.aws.amazon.com/sdk-for-javascript/v3/developer-guide/javascript_s3_code_examples.html
- GitHub, Workflow syntax for GitHub Actions. https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
- Tesseract.js project documentation. https://github.com/naptha/tesseract.js
- Mongoose documentation, Schemas and Validation. https://mongoosejs.com/docs/guide.html and https://mongoosejs.com/docs/validation.html

- Y. Huang et al., “LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking,” arXiv:2204.08387. https://arxiv.org/abs/2204.08387
- ScreenSync source repository contents reviewed for this report, local workspace state, 13 November 2026.